SYNOPSIS – Phase 2

Water Planning for Urban Household

Project Id: AIML PBL9

Member 1 (Team Leader):
Name: Aryan Arora
Class Roll No.: 35
University Roll No.: 2028666
Section: K
Group: G1

Member 2:
Name: Deepak Rana
Class Roll No.: 10
University Roll No.: 2028410
Section: K
Group: G1

Member 3:
Name: Aayush Bhala
Class Roll No.: 26
University Roll No.: 2028207
Section: K
Group: G1

TABLE OF CONTENTS

1. PROBLEM DESCRIPTION

Water is something we all use everyday but very few people actually keep track of how much they are consuming. In urban areas this problem is even bigger because the population density is high and water resources are limited. So the idea behind this project was to build a system that can predict how much water a particular household is going to use in a day based on some basic information about that household.

Things like how many people live in the house, whether they have a garden, how many appliances they use, what kind of building they stay in, what is the income level etc. – all of these factors affect water usage in some way. So we collected a dataset that has all this information and tried to train a machine learning model on it.

This kind of prediction can be really useful. For example if someone knows their household is likely to use a lot of water they can take steps to reduce it. And from a government or municipality side, if they can predict demand in advance they can plan water distribution more efficiently instead of dealing with shortages and wastage at the same time.

In Phase 2 we mainly focused on the EDA part, cleaning the data, doing some visualizations to understand it better, and then training regression models to predict the daily water usage in litres.

2. DATASET DESCRIPTION

We used a CSV file called urban_water_demand.csv for this project. We loaded it using pandas with pd.read_csv(). First thing we did after loading was check the shape of the dataset to see how many rows and columns it has. We printed the shape, column names, and data types to get a basic idea of what we are dealing with.

The target variable for this project is Daily_Liters_Used which is the amount of water consumed per day in litres. The features/input variables present in the dataset are listed below –

Column Name	Description
`Household_Size`	Number of people living in the household
`Seasonal_Index`	A numeric index representing the season
`Garden_Area`	Area of garden in the house (in sq. ft or similar)
`Avg_Temperature_C`	Average outside temperature in degree celsius
`Num_Appliances`	Number of water consuming appliances
`Income_Level`	Income category – Low / Medium / High (categorical)

`Building_Type`	Type of residential building (categorical)
`Daily_Liters_Used`	Target variable – daily water usage in litres

3. EXPLORATORY DATA ANALYSIS (EDA)

Before jumping into model training we spent some time exploring the data. This step is important because it helps in understanding the distribution of data, finding any missing values, checking correlations between features and identifying any outliers. We did the following things during EDA –

3.1 Checking Missing Values

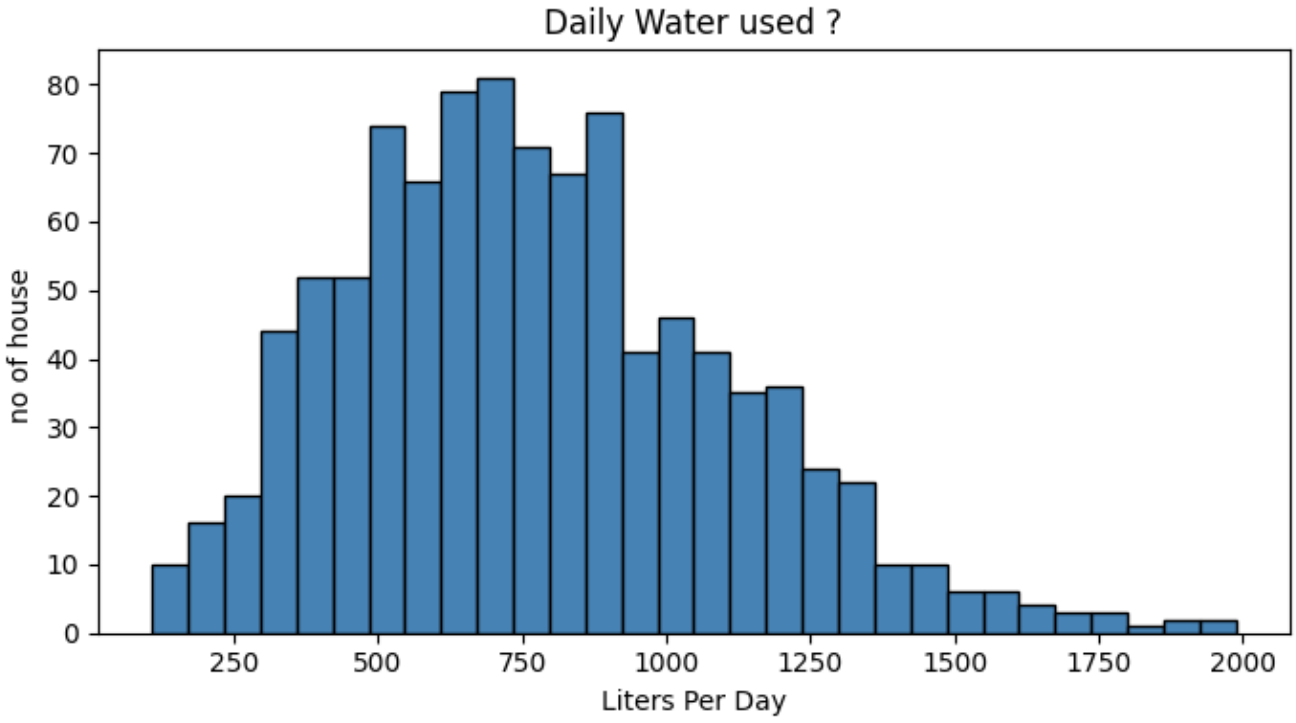
The first thing we checked was whether the dataset has any null values or not. We ran df.isnull().sum() and found that 5 columns had missing values – Household_Size, Seasonal_Index, Garden_Area, Avg_Temperature_C and Num_Appliances. We noted the counts and dealt with them in the preprocessing step.

3.2 Statistical Summary

We printed df.describe().round(2) to get a quick statistical summary. This gave us the count, mean, standard deviation, min, max and quartile values for each numeric column. Looking at this we could see the range of values for Daily_Liters_Used and how spread out the data is. Nothing looked too unusual at this stage.

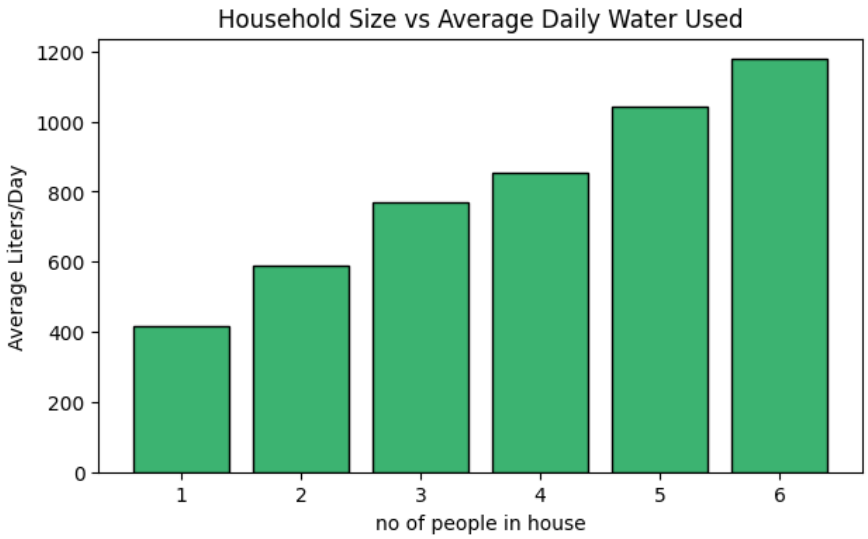
3.3 Distribution of Target Variable (Histogram)

We plotted a histogram of Daily_Liters_Used to see how the daily water usage is distributed across all households in the dataset. We used 30 bins, steelblue color, and added black edges to make the bars look cleaner. From the plot it looked like the data has a roughly normal distribution with most households using water in a moderate range.



3.4 Household Size vs Average Daily Water Usage

We made a bar chart where the x-axis shows household size (number of people, rounded to nearest integer) and y-axis shows the average litres used per day for that household size. As expected the average water usage goes up as the number of people increases. This confirmed that Household_Size is likely to be a useful feature for prediction.



4. DATA PREPROCESSING

After EDA we had a clear picture of what needed to be fixed before training. The main steps we did are explained below.

4.1 Handling Missing Values

For the 5 columns that had null values we decided to use imputation instead of dropping rows because dropping rows means losing data which we did not want to do. For Household_Size, Garden_Area and Num_Appliances we used the median because these are the kind of values where extreme outliers could skew the mean. For Seasonal_Index and Avg_Temperature_C we used the mean. After this step we ran df.isnull().sum() again and confirmed all values were 0.

4.2 Encoding Categorical Features

Income_Level and Building_Type are text/categorical columns and machine learning models cannot directly work with text data. So we used Label Encoding from sklearn to convert them into numeric values. We created

two separate LabelEncoder objects – le1 for Income_Level and le2 for Building_Type – and applied fit_transform on both. We also printed the first 5 rows after encoding just to verify that the conversion happened correctly.

4.3 Feature and Target Split

We separated the dataset into features (x) and target (y). x contains everything except Daily_Liters_Used and y contains only Daily_Liters_Used. There is a small naming inconsistency in the code – lowercase x was used for the split but uppercase X was printed. This was just a minor typo and did not affect the actual training.

4.4 Train-Test Split

We split the data using train_test_split with test_size=0.20 which means 80% of data is used for training and 20% is kept aside for testing. No random_state was set so results may vary slightly each run but that was fine for our purposes.

```
x_train, x_test, y_train, y_test = tts(x, y, test_size=0.20)
```

4.5 Feature Scaling (StandardScaler and MinMaxScaler)

We applied two types of scaling and compared them visually. StandardScaler scales data to mean=0 and std=1. MinMaxScaler scales data to be between 0 and 1. We plotted a 3-subplot comparison showing the distribution of Household_Size in original form, after StandardScaler and after MinMaxScaler. Interestingly the axis labels in this plot were written in Hinglish like 'StandardScaler ke Baad (mean=0, std=1)' which was just how it got coded during development.

Note: The actual model training was done on unscaled x_train directly. Scaling was mainly explored and visualized separately.

5. MODEL TRAINING

For this project we trained three regression models – Linear Regression, Lasso and Ridge. The reason we chose these three is that they are all linear models but differ in how they handle regularization. Training all three and comparing them gives a good idea of which approach works best for this dataset.

All three models were trained on the same training data without scaling and predictions were made on x_test.

5.1 Linear Regression

Linear Regression is the simplest and most straight forward regression model. It tries to find the best fit line through the training data by minimizing the sum of squared differences between actual and predicted values. We used the default LinearRegression() from sklearn without any hyperparameter tuning.

```
lr = LinearRegression()
lr.fit(x_train, y_train)
lr_pred = lr.predict(x_test)
```

5.2 Lasso Regression

Lasso (Least Absolute Shrinkage and Selection Operator) is basically Linear Regression with L1 regularization added to the loss function. The L1 penalty can push some feature coefficients all the way down to zero which means Lasso can also act as a feature selection method. This is useful when we suspect some features are not really contributing to the prediction.

```
lasso = Lasso()
lasso.fit(x_train, y_train)
las_predict = lasso.predict(x_test)
```

5.3 Ridge Regression

Ridge is similar to Lasso but uses L2 regularization instead of L1. The difference is that Ridge does not set coefficients to exactly zero – it just shrinks them. This makes it better when most features are contributing something to the output and you just want to avoid overfitting without eliminating any feature entirely.

```
ridge = Ridge()
ridge.fit(x_train, y_train)
rid_predict = ridge.predict(x_test)
```

6. EVALUATION METRICS USED

We used a combination of regression metrics and classification metrics to evaluate our models. The idea behind using both was to see performance from different angles.

6.1 R2 Score (Coefficient of Determination)

R2 score tells us how much of the variance in the actual values is being explained by the model. A score of 1 would mean perfect prediction and 0 means the model is no better than just predicting the mean. We used r2_score() from sklearn.metrics for this.

6.2 Mean Squared Error (MSE)

MSE gives the average of squared errors between actual and predicted values. Since errors are squared, larger mistakes get penalized more heavily. Lower MSE means better model performance.

6.3 Mean Absolute Error (MAE)

MAE gives the average of absolute errors. It is easier to interpret than MSE because it is in the same unit as the target variable (litres in our case). One thing to note in the code is that the variable lr_msc was first assigned the MSE value but then immediately reassigned with the MAE value on the next line. So in the final print statements both MSE and MAE end up printing the same value which is actually only the MAE. This was a small bug we noticed after running but the R2 scores were still printed correctly.

6.4 Accuracy and F1 Score

Since accuracy and F1 score are classification metrics they cannot directly be used for regression output. So we converted the predictions into categories using a custom function called categorize(). The logic was simple – if value is less than 500 it is Low, between 500 and 1000 it is Medium, and above 1000 it is High. Same categorization was applied to actual values and then accuracy_score and f1_score (weighted) were computed for all 3 models.

```
def categorize(val):
    if val < 500:    return 'Low'
    elif val < 1000: return 'Medium'
    else:            return 'High'
```

7. RESULTS AND VISUALIZATIONS

After training and evaluating all three models we visualized the results to get a better understanding of how the models are performing. The following plots were generated –

7.1 Actual vs Predicted Scatter Plot

A scatter plot was made comparing the actual y_test values against the lr_pred values from Linear Regression. A red dashed diagonal line (y = x) was also drawn on the same plot. If the model is predicting perfectly all the scattered points should fall exactly on this line. Our model showed points reasonably close to the diagonal which means the predictions are decent but not perfect, which is expected for a simple linear model.

7.2 Residuals vs Predicted Plot

The residuals (which are actual minus predicted values) were computed and plotted against the predicted values. Ideally residuals should be randomly scattered around the horizontal zero line with no visible pattern. If there is a pattern in the residuals it usually means the model is missing some underlying relationship in the data. In our case the residuals looked fairly random which is a good sign.

7.3 Error Distribution Histogram

We also plotted a histogram with a KDE (kernel density estimate) curve for the residuals. A good model should have residuals that are roughly normally distributed and centered around zero. If the error distribution is highly skewed it indicates some bias in the model. Our error distribution came out reasonably bell shaped.

7.4 Sample Comparison Table

A small comparison table of the first 10 test samples was printed showing three columns – Actual, Predicted and Error. Values were rounded to 1 decimal place. This gives a human readable way to see how far off the model is on individual predictions.

7.5 Confusion Matrix for All Three Models

Confusion matrices were plotted for Linear Regression, Lasso and Ridge side by side in a single figure with 3 subplots. The predictions were first converted to Low/Medium/High categories (same categorize function as used for accuracy/F1). sklearn's ConfusionMatrixDisplay was used to plot the matrices with Blues colormap. This gives a visual idea of which category the model is most confused about.

7.6 Model Saving with Pickle

After everything was done the trained Linear Regression model and the StandardScaler were saved together into a file called model.pkl using Python's pickle library. We stored both inside a dictionary with keys 'model' and 'scaler' so they can be loaded together easily later.

```
save_data = {'model': lr, 'scaler': scaler}
with open('model.pkl', 'wb') as f:
```

```
    pickle.dump(save_data, f)
```

We then loaded the file back and made a sample prediction for a new household (4 people, 80 sq ft garden, 5 appliances etc.) to verify the model was saved and loaded correctly. The output printed as 'Predicted Water Usage: X Liters/day'.

8. REFERENCES

Books

- Aurélien Géron – Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow
- Yashavant Kanetkar – Let Us Python

Online Resources

- scikit-learn Documentation – https://scikit-learn.org/stable/
- pandas Documentation – https://pandas.pydata.org/docs/
- Towards Data Science – https://towardsdatascience.com
- TutorialsPoint – https://www.tutorialspoint.com